Object Detection & Localization

Week 14 — Session 2: Single-shot detectors (YOLO) and measuring them (mAP)

Practical Image Processing

Lecture 14.2 ▪ 60–70 min

Recap of Session 1

- Detection = classify *and* localize — a variable number of boxes.
- A box is 4 numbers; localization is regression on them.
- **Sliding window** sweeps a classifier everywhere over an image pyramid — correct but slow.
- **IoU** scores overlap; **NMS** collapses duplicate boxes into final detections.

Today's job

Replace the brute-force sweep with *one* CNN forward pass (YOLO), then learn to *measure* a detector honestly with mAP — and detect the DocScan digit strip.

Roadmap — Session 2

From proposals to single-shot

How YOLO sees an image

Measuring a detector: mAP

Code: IoU and NMS from scratch

In-class exercise

Wrap-up

From proposals to single-shot

First fix: propose, don't sweep

- Sliding window tests *every* location and scale — most are empty background.
- **Region proposals** (e.g. **selective search**) suggest a few thousand "objecty" boxes first.
- Run the CNN only on those — huge saving vs. hundreds of thousands of windows.

The two-stage family: R-CNN → Fast → Faster R-CNN

Stage 1: *where might objects be?* Stage 2: *what is in each proposal + refine its box.*

Key idea

Two-stage detectors are accurate but still run a classifier once per proposal.

A detector timeline (intuition, not dates to memorize)



- Each step removes redundant work: fewer windows, then shared features, then one pass.
- The *ideas* stack — IoU, NMS, and anchors carry all the way across this line.

Key idea

The field's whole arc is "do the same detection, but cheaper."

Second fix: skip stages — single-shot

- **Single-shot** detectors (**YOLO**, **SSD**) do it all in *one* forward pass.
- No separate proposal step: the CNN outputs boxes *and* classes directly.
- Fast enough for video — "You Only Look Once."



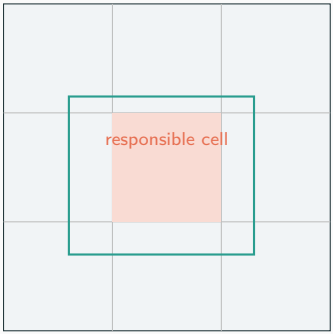
Key idea

Trade a little accuracy for a lot of speed — and NMS still cleans up at the end.

How YOLO sees an image

The grid idea

- Divide the image into an $S \times S$ grid (here 3×3).
- The cell that contains an object's *centre* is responsible for detecting it.
- Each cell predicts a few boxes relative to itself, plus class scores.



What each cell predicts

For every cell, for each of its B boxes:

Prediction	Meaning
t_x, t_y	box centre, *offset inside the cell* (0–1)
t_w, t_h	box width/height (relative to the image or an anchor)
objectness p_o	"is there *any* object here?" confidence
class scores $p_1..p_C$	"if there is, which class?"

Box confidence used for NMS

score $= p_o \times p_{\text{class}}$ — objectness times class probability

The output tensor, counted

One grid, one tensor

output shape $= S \times S \times (B \cdot 5 + C)$

- The $5 = (t_x, t_y, t_w, t_h, p_o)$ per box; C = one class score set per cell.
- DocScan example: S=7, B=2 boxes, C=2 classes (document, digit-strip):

$$7 \times 7 \times (2 \cdot 5 + 2) = 7 \times 7 \times 12 = 588 \text{ numbers.}$$

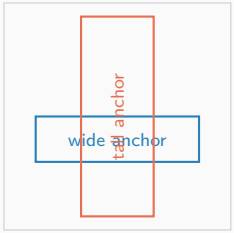
- One CNN forward pass produces *all* of it at once — no sweeping.

Key idea

Detection becomes "predict one fixed-size tensor," then decode + NMS it.

Anchor boxes: priors for shape

- Objects have typical shapes: a digit strip is *wide*, a dial is *square*.
- **Anchors** are a few preset box shapes per cell; the net predicts a small *correction* to an anchor, not a box from scratch.
- Easier to learn a nudge than an absolute size — and different anchors specialize in different shapes.



Key idea

An anchor is an educated guess about a box's shape; the net only refines it.

Training a single-shot detector (intuition)

- Assign each ground-truth object to the cell/anchor with best **IoU**.
- Loss has three parts, summed:
  - *box* loss (regression) for the responsible box;
  - *objectness* loss ($p_o \to 1$ where an object is, $\to 0$ elsewhere);
  - *class* loss (cross-entropy) for the responsible cell.
- Most cells are background $\Rightarrow$ weight objectness carefully or the net predicts "nothing."

Math

$$L = \lambda_{\text{box}} L_{\text{box}} + L_{\text{obj}} + L_{\text{cls}}$$

At inference: decode, then NMS

1. Forward pass $\rightarrow$ the $S \times S \times (B \cdot 5 + C)$ tensor.
2. For each box: score $= p_o \times p_{\text{class}}$; drop scores below a threshold.
3. Convert cell-relative (t_x, t_y, t_w, t_h) to absolute pixel boxes.
4. Run **NMS** per class — exactly Session 1's algorithm.

✔ Good practice

The detector proposes; NMS disposes. Session 1's two tools are still doing the final cleanup.

Two different knobs (do not confuse them)

Confidence threshold

Applied *before* NMS to each box's score = $p_o \times p_{\text{class}}$. Raise it $\rightarrow$ fewer, more certain boxes (fewer false positives, more misses).

NMS IoU threshold τ

Applied *during* NMS between surviving boxes. Raise it $\rightarrow$ more overlaps allowed to coexist (more duplicates in crowds).

✗ Pitfall

"Too many boxes" has two cures — raise the *confidence* threshold, or lower the *NMS* threshold. They fix different problems; know which one you are turning.

Measuring a detector: mAP

A detection is right only if the box is right

- A prediction counts as a **true positive** if its IoU with a ground-truth box $\geq$ a threshold (usually 0.5) *and* the class matches.
- Otherwise it is a **false positive**; a missed ground-truth is a **false negative**.
- Only *one* prediction may claim each ground-truth box — extra ones are false positives.

Precision & recall

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}$$

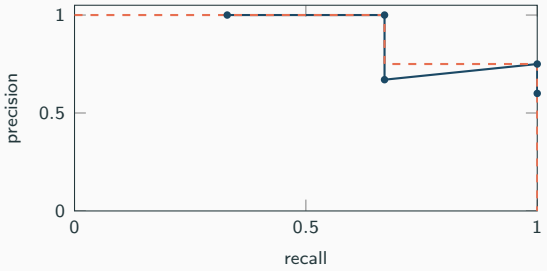
Precision & recall, worked

5 predictions ranked by score; 3 ground-truth objects; IoU≥ 0.5 = a hit.

Rank	Result	cum TP	Precision	Recall
1	TP	1	$1/1 = 1.00$	$1/3 = 0.33$
2	TP	2	$2/2 = 1.00$	$2/3 = 0.67$
3	FP	2	$2/3 = 0.67$	0.67
4	TP	3	$3/4 = 0.75$	$3/3 = 1.00$
5	FP	3	$3/5 = 0.60$	1.00

As we accept lower-scored boxes, recall rises but precision tends to fall — the classic trade-off.

The precision–recall curve and AP



- **Average Precision (AP)** = area under the (interpolated) PR curve.
- Coral: the "max precision to the right" interpolation used in practice.
- **mAP** = mean AP across all classes.

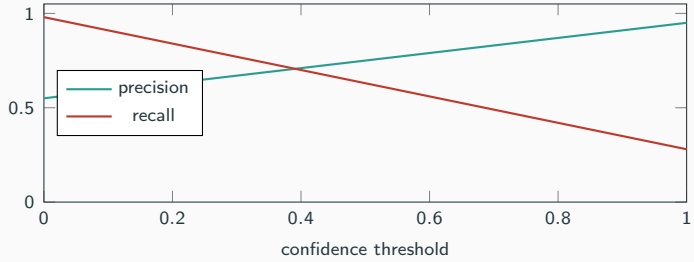
mAP: the one number people quote

- Compute AP *per class*, then average $\rightarrow$ **mAP** (mean Average Precision).
- `mAP@0.5`: hits need IoU ≥ 0.5. `mAP@[.5:.95]`: average over many IoU thresholds (stricter, COCO-style).
- Higher mAP = better boxes *and* better labels, in one score.

✕ Pitfall

A high *classification* accuracy says nothing about box quality. Detection must be judged with IoU-aware metrics like mAP — never plain accuracy.

The precision–recall trade-off, in one picture



- Raise the confidence threshold: precision climbs (fewer false positives), recall falls (more misses).
- mAP integrates over this whole sweep, so it does not hinge on one hand-picked threshold.

Code: IoU and NMS from scratch

IoU in NumPy

```python
def iou(a, b):                          # boxes as (x1, y1, x2, y2)
    ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
    ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
    iw, ih = max(0, ix2 - ix1), max(0, iy2 - iy1)   # clamp at 0
    inter = iw * ih
    area_a = (a[2]-a[0]) * (a[3]-a[1])
    area_b = (b[2]-b[0]) * (b[3]-b[1])
    return inter / (area_a + area_b - inter + 1e-9)
```

- The `max(0, ...)` clamp is the whole trick — non-overlapping boxes give 0.
- `+ 1e-9` avoids a divide-by-zero on degenerate (zero-area) boxes.
- This matches the by-hand result: $\text{iou}((1,1,5,4),(3,2,7,6)) = 0.167$.

NMS in NumPy

```python
def nms(boxes, scores, tau=0.5):
    order = sorted(range(len(scores)),           # high score first
                   key=lambda i: scores[i], reverse=True)
    keep = []
    while order:
        i = order.pop(0)                         # take the best
        keep.append(i)
        order = [j for j in order                # drop overlaps
                 if iou(boxes[i], boxes[j]) < tau]
    return keep
```

- Greedy: keep the top box, remove everything that overlaps it past τ, repeat.
- Runs *per class* — never suppress a dial with a meter.
- Returns the indices of the surviving boxes — the final detections.

Localizing the digit strip (the DocScan tie-in)

```
# score every sliding window, then clean up with NMS
boxes, scores = [], []
for (x, y, w, h) in slide(img, win=(120, 40), stride=8):
    s = strip_score(img[y:y+h, x:x+w])   # "looks like a digit strip?"
    if s > 0.6:
        boxes.append((x, y, x+w, y+h)); scores.append(s)
keep = nms(boxes, scores, tau=0.4)       # collapse the cluster
strip = boxes[keep[0]]                    # feed this crop to Week 13's reader
```

- Same pipeline, tiny scale: sweep $\rightarrow$ score $\rightarrow$ NMS $\rightarrow$ one clean box.
- That one box is exactly the crop Week 13's digit CNN expects.

In-class exercise

In-class exercise (10 min)

Reason about detectors and their scores

1. A YOLO grid is 7×7 with $B{=}3$ boxes and $C{=}5$ classes. What is the output tensor size?
2. Predictions ranked TP, FP, TP with 2 ground-truth objects. Give precision & recall after each.
3. Why must a detector be scored with mAP rather than plain classification accuracy?
4. Your detector reports one meter as three boxes. Which knob do you reach for, and how?

Debrief

- (1) $7 \times 7 \times (3 \cdot 5 + 5) = 7 \times 7 \times 20 = 980$ numbers.
- (2) TP: P=1.00, R=0.5; FP: P=0.5, R=0.5; TP: P=0.67, R=1.0.
- (3) Accuracy ignores *where* the box is; mAP rewards correct class *and* IoU-tight boxes, and balances precision vs recall over all thresholds.
- (4) **NMS**: the IoU threshold τ is too high — lower it so the duplicate boxes get suppressed.

Key idea

You can now size a detector's output, read its PR numbers, and debug its boxes.

Wrap-up

Two-stage vs single-shot — an honest comparison

	Two-stage (Faster R-CNN)	Single-shot (YOLO/SSD)
Speed	slower	real-time
Accuracy	typically higher	very close, improving
Small objects	handles better	historically weaker
Simplicity	pipeline of parts	one network

Key idea

Need real-time on a phone (DocScan)? Single-shot. Need every tiny object at any cost? Two-stage.

Pitfalls when building detectors

✕ Watch out

- Judging a detector by accuracy instead of mAP — boxes can be badly placed yet "accurate."
- Running NMS across classes, or forgetting it entirely (duplicate boxes everywhere).
- Ignoring the background/foreground imbalance — the net learns to predict "nothing."
- Anchor shapes mismatched to your objects — give a wide digit strip a wide anchor.

What you should now be able to do

- Explain the path from sliding windows $\rightarrow$ region proposals $\rightarrow$ single-shot detectors.
- Describe YOLO's grid, anchors, objectness, and its $S \times S \times (B \cdot 5 + C)$ output tensor.
- Decode + NMS a detector's output into final boxes.
- Score a detector with precision, recall, AP, and mAP — and read a PR curve.

This week's deliverable

Due before the capstone finale (see `project/`)

1. Implement `iou` and `nms`; verify IoU on the by-hand boxes ($= 0.167$) and disjoint boxes ($= 0$).
2. Run the sliding-window demo: localize the digit strip in a synthetic meter image, then NMS.
3. Feed the localized crop to a digit reader (reuse Week 13) — report the read value.
4. Compute precision/recall for a small set of detections vs. ground truth; sketch the PR curve.
5. Reflect: how does "find the strip, then read it" complete the DocScan pipeline?

Find it, then read it.

One CNN sweep, decoded into boxes, cleaned by NMS, judged by mAP — detection is Week 13 aimed at “where.”